LUCRAREA 2

IMPLEMENTAREA ALGORITMILOR PARALELI. CONDIȚII DE PARALELISM.

Implementarea paralelă a aplicațiilor sau a algoritmilor se bazează pe o serie de noțiuni și concepte specifice calculului paralel precum și pe o serie de resurse hardware și software necesare. Acestea vor fi analizate în cadrul cursului, urmând ca pe parcursul orelor de aplicații să utilizăm aceste noțiuni și concepte pentru implementarea paralelă a unor probleme tipice.

În acest context ne vom baza pe noțiuni precum sisteme de operare multitasking, sisteme de operare multithreading, procese, threaduri, elemente de sincronizare, resursele/componentele unui sistem de calul (CPU, RAM, I/O), urmând ca în continuare să studiem o serie de metode și modele de implementare a algoritmilor paraleli.

1. CONDIȚII DE PARALELISM

Pentru ca mai multe segmente de program (procese, thread-uri, instucţiuni) să poată fi executate concurent este necesar ca fiecare segment să fie independent de toate celelalte segmente.

Pentru exprimarea relațiilor de dependenţă între segmentele unui program se foloseşte un graf direcţionat, numit graful de dependenţă al programului (***program dependence graph***). Nodurile grafului de dependenţă corespund segmentelor programului (care, în acest context vor fi numite task-uri), iar arcele direcţionate reprezintă relaţiile de dependenţă între task-uri. Analiza grafului de dependenţă al programului permite identificarea posibilităţilor de execuţie paralelă a programelor.

Dependenţele existente în execuţia task-urilor pot fi de mai multe categorii, şi anume: dependenţe de date, de control şi de resurse.

Fie două task-uri T1 şi T2, unde T2 urmează lui T1 în ordinea programului. Dependenţele de date între aceste task-uri pot fi următoarele:

(1) Dependenţa prin succesiune (***flow dependence***). Un task T2 este dependent de taskul T1, dacă cel puţin o variabilă actualizată de T1 este folosită ca variabilă de intrare de către T2. Dependenţa prin succesiune se reprezintă astfel: T1 → T2.

(2) Antidependenţa (***antidependence***). Un task T2 este antidependent de task-ul T1, dacă T2 urmează lui T1 ca ordine a programului, şi o ieşire a lui T2 actualizează o intrare a task-lui T1. Antidependenţa se reprezintă printr-o săgeată intersectată de o bară verticală: T1 |→ T2.

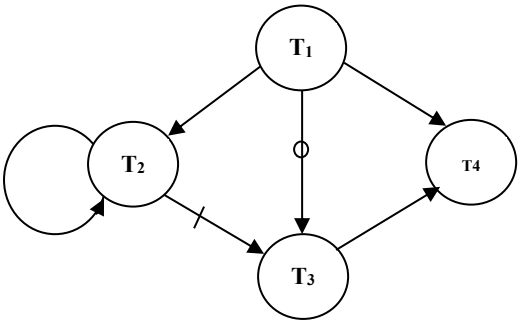
(3) Dependenţa prin ieşire (***output dependence***). Două task-uri T1 şi T2 prezintă dependenţă de iesire, dacă ele actualizează aceeaşi variabilă de ieşire. T1 o→ T2 indică dependenţa de ieşire între task-urile T1 şi T2.

(4) Dependenţa necunoscută (***unknown dependence***). Relaţia de dependentă între două task-uri nu poate fi determinată dacă ele folosesc în comun (partajează) tablouri de variabile şi:

- Indicele unei variabile dintr-un tablou este el însuşi o variabilă indexată.
- Indicele unei variabile dintr-un tablou nu conţine variabila de control a buclei de execuţie sau variază nelinear cu aceasta.

- O variabilă este apelată de mai multe ori, cu indici care depind în mod diferit de variabila de control a buclei.

Exemplu de graf al dependențelor:



1.1 CONDIŢIILE DE PARALELISM ALE LUI BERNSTEIN

Fie I_i mulţimea variabilelor de intrare ale unui task T_i, şi O_i mulţimea variabilelor de ieşire ale aceluişi task T_i. Variabilele de intrare sunt, în general, operanzii extraşi din registre sau memorie, iar variabilele de ieşire sunt datele rezultate care vor fi înscrise în registre sau locaţii de memorie. Se consideră două task-uri T1 şi T2, cu mulţimile variabilelor de intrare I1 şi I2, respectiv, şi mulţimile variabilelor de ieşire O1 şi O2. Aceste două taskuri pot fi executate în paralel, ceea ce se notează cu T1 || T2, dacă sunt independente, deci nu prezintă nici un conflict în execuţie. Condiţiile de execuţie paralelă pot fi scrise astfel:

$I1 \cap O2 = \Phi$

$I2 \cap O1 = \Phi$

$O1 \cap O2 = \Phi$.

Aceste condiţii sunt cunoscute sub numele de condiţiile lui Bernstein. Mulţimea I_i se mai numeşte domeniul task-ului T_i, iar mulţimea O_i rangul acestuia. Condiţiile lui Bernstein implică faptul că două procese pot fi executate în paralel dacă sunt independente. Execuţia paralelă a două task-uri produce acelaşi rezultat, indiferent dacă sunt executate secvenţial în orice ordine, sau în paralel. Acest lucru este posibil numai dacă nici o ieşire a unui proces nu este intrare a celuilalt şi ele nu scriu (nu modifică) aceleaşi variabile, în registre sau memorie.

În general, o mulţime de task-uri T1, T2, …., Tk, pot fi executate în paralel dacă condiţiile lui Bernstein sunt satisfăcute pentru fiecare pereche de task-uri., adică:

T1 || T2 || T3 ||….||Tk, dacă şi numai dacă Ti || Tj, pentru orice $i \neq j$.

Relaţia de paralelism este comutativă, adică T1 || T2, implică T2 || T1, dar nu este tranzitivă, adică Ti || Tj, şi Tj || Tk, nu garantează Ti || Tk. De exemplu, se poate ca T1 || T5 şi T5 || T2, dar T1 # T2 , unde notaţia # reprezintă faptul că cele două taskuri nu pot fi executate în paralel.

Relaţia de paralelism este, de asemenea, o relaţie asociativă, de vreme ce ordinea în care sunt executate task-urile nu introduce diferenţe în rezultatul obţinut (variabilele de ieşire).

Violarea oricăreia din condiţiile lui Bernstein inhibă paralelismul între două task-uri.

De asemenea, secţiunile de program care depind de condiţii din timpul rulării (run-time), ca de exemplu salturile condiţionate (de ex. instrucţiunile IF), nu pot fi transformate într-o formă paralelă.

În general, dependenţa de date, de control sau de resurse, inhibă execuţia paralelă a task-urilor.

Aşa cum s-a precizat de la începutul acestui paragraf, dependenţa se poate analiza la nivelul cel mai de jos de reprezentare a unui program, adică la nivelul instrucţiunilor, cod maşină sau de nivel înalt, sau poate fi generalizată pentru segmente de cod implementate ca procese, thread-uri, task-uri, sau chiar la nivel de programe

Scopul analizei dependenţelor între task-urile componente ale unui program, este de a identifica posibilităţile de paralelizare a programului. Prin paralelizarea unui program secvenţial se înţelege transformarea acestuia într-o formă executabilă în paralel.

Paralelizarea unui program secvenţial poate fi făcută manual, de către programator, numindu-se paralelizare explicită, sau poate fi executată automat, de către compilator, numindu-se paralelizare implicită. În continuare vom aborda doar varianta paralelizării explicite.

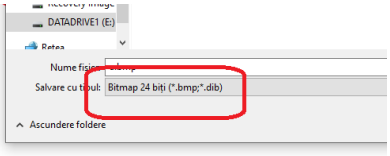
Operaţia de paralelizare nu este o operaţie banală. În ambele abordări, paralelizarea începe cu partiţionarea (divizarea) programului în unităţi de calcul (segmente), de diferite dimensiuni (nivele de granularitate), inclusiv până la nivelul de instrucţiune, care pot fi candidate pentru a fi executate în paralel. După partiţionarea programului, detecţia paralelismului între segmentele în care acesta a fost divizat se realizează prin testarea diferitelor relaţii de dependenţă, ca cele prezentate mai sus [1].

În continuare vom considera problema procesării unei imagini aflate într-un director cu salvarea rezultatului într-un alt director. Procesarea unei imagini constă în: citirea listei de imagini din director (Task ***A***) încărcarea imaginii din fișier (Task ***B***), transformarea imaginii din imagine color în imagine cu nivele de gri (Task ***C***), și la final salvarea imaginii rezultate în directorul destinație (Task ***D***) :

```
static void Main(string[] args)
{
    //atentie la calea de pe disk unde se afla imaginile!!!
    string fileName = @"C:\Studenti\SourceImages\a.bmp"; //Task A

    using (Bitmap image = new Bitmap(fileName)) //Task B
    {
        for (int x = 0; x < image.Width; x++)  //Task C
        {
            for (int y = 0; y < image.Height; y++)
            {
                Color pixelColor = image.GetPixel(x, y);
                int gray = (pixelColor.R + pixelColor.G + pixelColor.B) / 3;
                Color newColor = Color.FromArgb(gray, gray, gray);
                image.SetPixel(x, y, newColor);
            }
        }
        //atentie la calea de pe disk unde se salveaza imaginile!!!
        image.Save(@"C:\Studenti\FinalImages\a.bmp"); //Task D
    }
}
```

Atentie – Pentru lucrul cu imagini recomandam crearea unei imagini noi in Paint si salvarea in formatul BMP si 24bpp in folderul setat in codul sursa al programelor. Nu preluati imagini din surse externe (ex. Internet), acestea pot avea formate incompatibile cu libraria/clasa Bitmap:



EXERCIȚIUL 1:

Analizați dependența de date între taskurile identificate anterior. Identificați și desenați graful dependențelor între taskuri. O astfel de problemă se poate paraleliza?

În continuare vom considera problema procesării unui set de imagini aflate într-un director cu salvarea rezultatului într-un alt director, procesarea făcându-se conform task-urilor definite anterior (A - D):

```
static void Main(string[] args)
{
    //atentie la calea de pe disk unde se afla imaginile!!!
    string sourceDirectory = @"C:\Studenti\SourceImages";
    string finalDirectory = @"C:\Studenti\FinalImages";
    string[] fileEntries = Directory.GetFiles(sourceDirectory, "*.bmp").OrderBy(x =>
x).ToArray(); //Task A
    foreach (string fileName in fileEntries)
    {
        using (Bitmap image = new Bitmap(fileName)) //Task B
        {
            for (int x = 0; x < image.Width; x++)  //Task C
            {
                for (int y = 0; y < image.Height; y++)
                {
                    Color pixelColor = image.GetPixel(x, y);
                    int gray = (pixelColor.R + pixelColor.G + pixelColor.B) / 3;
                    Color newColor = Color.FromArgb(gray, gray, gray);
                    image.SetPixel(x, y, newColor);
                }
            }
            image.Save(Path.Combine(finalDirectory, Path.GetFileName(fileName))); //Task D
        }
    }
}
```

EXERCIȚIUL 2:

Analizați dependența de date între taskurile identificate anterior. Identificați și desenați graful dependențelor între taskuri. O astfel de problemă se poate paraleliza?

Pentru a trece de la o implementare serială (secvențială) la una paralelă, trebuie create unul sau mai multe threaduri și sarcina de calcul inițială trebuie distribuită threadurilor. O posibilă implementare poate să conțină o listă partajată de sarcini de calcul si fiecare thread preia (sau i se distribuie) sarcini de calcul.

```
//atentie la calea de pe disk unde se afla imaginile!!!
public static string sourceDirectory = @"C:\Studenti\SourceImages";
public static string finalDirectory = @"C:\Studenti\FinalImages";
public static string[] fileEntries; //lista partajata
.....
static void Main(string[] args)
{
```

```
    fileEntriesș = Directory.GetFiles(sourceDirectory, "*.bmp").OrderBy(x => x).ToArray();
//Task A

    Thread th1 = new Thread(ThreadFunction);
    Thread th2 = new Thread(ThreadFunction);
    th1.Start(0); //parametru – posibil mecanism de calcul partiție pentru threadul curent
    th2.Start(1);
    th1.Join();
    th2.Join();

}
```

EXERCIȚIUL 3:

Implementați problema de mai sus folosind două threaduri. Implementați problema de mai sus folosind trei threaduri. Implementați problema de mai sus folosind un număr N de threaduri, unde N este dat ca parametru de intrare la rularea aplicației.

O implementare paralelă poate fi considerată ca fiind de succes dacă timpul de execuție a algoritmului paralel este mai mic decât timpul de execuție a implementării seriale, aceasta corespunzând unei accelerări paralele mai mare ca 1. Pentru evaluarea timpului de execuție se poate utiliza un cronometru ca de exemplu:

```
Stopwatch timer = new Stopwatch();
timer.Start();

..
//cod de evaluat
..

timer.Stop();

Console.WriteLine("Elapsed = {0} (ms)", timer.Elapsed.TotalMilliseconds);
Console.ReadLine();
```

EXERCIȚIUL 4:

Evaluați timpul de execuție a implementării seriale pentru problema de mai sus. Evaluați timpul de execuție a implementării paralele. Evaluați variația timpului de execuție paralelă pentru un număr de threaduri N=1, 2, 3,.., 20 (conform implementării din exercițiul 3 de mai sus). Calculați accelerarea paralelă (***speedup***-ul) $S=T_{serial}/T_{paralel}$

2. PLANIFICAREA DINAMICĂ A TASK-URILOR

Planificarea dinamică a task-urilor implică distribuția sarcinilor de calcul procesoarelor în cursul execuției, spre deosebire de planificarea statică ce alocă sarcinile de calcul la începutul execuției aplicației sau prin design (prin modul cum este implementată aplicația). Este posibil ca planificarea dinamică să crească timpul de execuție a algoritmului paralel, dacă timpul consumat pentru decizia de planificare este mai mare decât cel câștigat prin

amplasarea diferitelor task-uri în timpul execuției. Măsura succesului planificării dinamice o constituie reducerea netă a timpului de execuție paralelă.

Programatorului îi revine sarcina de a prevedea mecanisme prin care se planifică procesele în diferitele procesoare, pentru asigurarea încărcării echilibrate a procesoarelor. Planificarea dinamică se realizează printr-unul sau mai multe module de program, care se execută odată cu toate celelalte sarcini ale programului paralel şi atribuie procesele active procesoarelor sistemului. Este de dorit ca reducerea timpului de execuţie paralelă printr-o bună echilibrare dinamică a încărcării procesoarelor să fie mai importantă decât timpul consumat pentru execuţia operaţiilor de planificare dinamică.

Un exemplu de planificare dinamică constă în crearea unei liste de task-uri/sarcini de calcul din care thread-urile care au finalizat un task să extragă un nou task. Desigur accesul la lista de task-uri disponibile trebuie să fie unul mutual exclusiv, adică un task trebuie să fie preluat doar de un thread, să nu existe situații în care un același task este preluat de două sau mai multe threaduri, lucru ce poate duce la execuții ineficiente sau la erori.

EXERCIȚIUL 4:

Modificați programul astfel încât să implementați alocarea dinamică a taskurilor (imaginile de procesat) către threaduri. Comparați timpul de execuție cu cel al implementării de la Exercițiul 3.

EXERCIȚIUL 5:

Modificați setul de imagini astfel încât să testați eficiența implementărilor paralele de la exercițiile 3 și 4. Testați pe imagini mici (ex. 100x100px), medii (ex. 500x500px) sau mari (2000x2000px).

BIBLIOGRAFIE

[1]. PRINCIPIILE CALCULULUI PARALEL, Felicia Ionescu, EDITURA TEHNICĂ, București, 1999

[2]. C# Programming Guide – Thread Class https://docs.microsoft.com/en-us/dotnet/api/system.threading.thread?view=netframework-4.8